CV WEEK2 LECTURE NOTES

What Features Represent?

In computer vision, when we talk about **"features"** we mean **distinctive, informative patterns in an image** that can be detected, described, and compared.

They are the building blocks for tasks like recognition, matching, and tracking.

- **Corners:** Points where intensity changes in two directions (e.g., Harris corners).
- **Edges:** Boundaries between regions of different brightness or texture.
- **Blobs/Keypoints:** Regions that stand out compared to their surroundings (e.g., SIFT, ORB).
- **Descriptors:** Numerical vectors that describe the local neighbourhood around a feature (e.g., SIFT descriptor, ORB binary string).

Why Features Matter?

- **Uniqueness:** Features should be distinctive enough to be matched across different images.
- **Repeatability:** The same feature should be detectable even if the image changes (scale, rotation, lighting).
- **Compact Representation:** Instead of comparing entire images pixel by pixel, we compare sets of features.

Examples:

- **Harris Corners:** Detects corners by analyzing intensity changes.
- **SIFT:** Finds scale and rotationinvariant keypoints and describes them with robust vectors.
- **ORB:** Fast alternative to SIFT, producing binary descriptors for efficiency.

Why Feature Detection?

- **Definition:** Identifying distinctive points in images (corners, blobs, edges) that can be matched across views.
- **Purpose:** Provides anchor points for recognition, tracking, and stitching.
- **Applications:**

- Object recognition (finding known patterns in new images)
- Motion tracking (following features across frames)
- Panorama stitching (aligning overlapping images)

Harris Corner Detection

- **Concept:** Corners are points where intensity changes in multiple directions.
- **Method:** Uses gradient information to compute a “corner response” score.
- **Pros:**
  - Simple and effective for detecting corners.
  - Good for rigid structures (buildings, checkerboards).
- **Cons:**
  - Not invariant to scale or rotation.
  - Sensitive to image transformations.
- **Exercise Outcome:** See red dots marking corners in the sample image.

SIFT (ScaleInvariant Feature Transform)

- **Concept:** Detects and describes keypoints robust to scale and rotation.
- **Pipeline:**
  1. Build Gaussian scale space.
  2. Detect extrema (Difference of Gaussians).
  3. Assign orientation.
  4. Create descriptor vector (128dimensional).
- **Pros:**
  - Very robust, widely used in research and industry.
  - Handles changes in scale, rotation, and illumination.
- **Cons:**
  - Computationally heavy.
  - Historically patented (now free in OpenCV ≥ 4.4).

- **Exercise Outcome:** See rich keypoints drawn with size and orientation.

ORB (Oriented FAST and Rotated BRIEF)

- **Concept:** Combines FAST keypoint detector with BRIEF descriptor, adds orientation.
- **Pros:**
    - Free, fast, efficient.
    - Rotation invariant.
- **Cons:**
    - Less robust than SIFT for large scale changes.
- **Exercise Outcome:** See green keypoints drawn quickly, suitable for realtime tasks.

Exercise: Detect Keypoints in Images

- **Goal:** Compare Harris, SIFT, and ORB on different images.
- **Learning Point:**
    - Harris finds corners but misses scale/rotation changes.
    - SIFT finds fewer but more reliable keypoints.
    - ORB finds many fast keypoints, good for realtime applications.

Mini Project: Panorama Stitching

- **Workflow:**
    1. Detect features (ORB).
    2. Match features between overlapping images.
    3. Estimate homography (geometric transformation).
    4. Warp and stitch images into a panorama.
- **Key Concepts:**
    - **Feature Matching:** Using descriptors to find correspondences.
    - **Homography:** Transformation mapping one image plane to another.
    - **RANSAC:** Robust estimation to handle outliers.
- **Outcome:** See two images stitched together into a panorama.

Image Matching

Why Image Matching?

- **Definition:** Image matching is the process of finding correspondences between features in two or more images.
- **Purpose:** Enables recognition of objects, logos, or scenes across different images.
- **Applications:**
    - Logo detection in advertising or media.
    - Object recognition in robotics and surveillance.
    - Image retrieval in search engines.

Feature Descriptors

- **Concept:** A descriptor encodes the local neighbourhood around a detected keypoint into a numerical vector.
- **Examples:**
    - **SIFT (ScaleInvariant Feature Transform):** Robust to scale and rotation.
    - **ORB (Oriented FAST and Rotated BRIEF):** Fast, efficient, free alternative.
    - **SURF (Speeded Up Robust Features):** Faster than SIFT, but patented.
- **Role:** Descriptors allow comparison of features across images — two keypoints are considered a match if their descriptors are similar.

FLANN (Fast Library for Approximate Nearest Neighbours)

- **Concept:** FLANN is a library optimized for fast matching of highdimensional feature descriptors.
- **Pros:**
    - Efficient for large datasets.
    - Scales well with many features.
- **Cons:**
    - Approximate — may miss exact matches.
    - Requires parameter tuning for best results.

- **Key Technique:** Often combined with **Lowe's ratio test** to filter out weak or ambiguous matches.

Exercise: Match Logos Across Images

- **Goal:** Replace logo.jpg and scene.jpg with different images.
- **Learning Point:**
    - Observe how many matches are found.
    - Evaluate accuracy — are matches clustered around the logo or scattered incorrectly?
- **Discussion:** Can compare ORB vs SIFT descriptors for robustness and speed.

Mini Project: Logo Detector

- **Workflow:**
    1. Detect features in both logo and scene.
    2. Match descriptors using FLANN.
    3. Apply Lowe's ratio test to keep good matches.
    4. Estimate **homography** (geometric transformation) between logo and scene.
    5. Draw bounding box around detected logo.
- **Key Concepts:**
    - **Homography:** Maps points from one image plane to another.
    - **RANSAC:** Robust estimation method to handle outliers in matches.
- **Outcome:** See the logo localized in the scene image.

Tips

- Emphasize the **difference between detection and description**: keypoints vs descriptors.
- Show how **matching quality** depends on descriptor choice and ratio test threshold.
- Connect exercises to **realworld applications**: brand monitoring, AR, visual search.
- Experiment with different logos and cluttered scenes to test robustness.

Object Tracking Basics

Why Object Tracking?

- **Definition:** Object tracking is the process of following a moving target across consecutive video frames.
- **Purpose:** Essential for applications like surveillance, sports analytics, autonomous driving, and human–computer interaction.
- **Challenge:** Objects may change appearance, get occluded, or move quickly, so robust tracking methods are needed.

Optical Flow

- **Concept:** Optical flow estimates the motion of pixels between consecutive frames.
- **Types:**
    - **Dense Optical Flow:** Computes motion vectors for *every pixel*. Useful for motion visualization.
    - **Sparse Optical Flow:** Tracks only selected keypoints. More efficient, often used in realtime systems.
- **Key Idea:** Motion is inferred from changes in brightness patterns across frames.
- **Limitations:** Sensitive to large displacements, lighting changes, and occlusions.

KLT (Kanade–Lucas–Tomasi) Tracker

- **Concept:** A widely used tracker based on sparse optical flow.
- **Steps:**
    1. Detect “good features to track” (usually corners).
    2. Use Lucas–Kanade optical flow to follow these features frame by frame.
- **Pros:**
    - Efficient and relatively simple.
    - Works well for moderate motion and stable lighting.
- **Cons:**
    - Struggles with occlusion (object hidden) or large motion jumps.
    - Requires reinitialization if features are lost.

Exercise: Track Moving Ball in Video

- **Goal:** Apply KLT tracker to a video (ball.mp4).
- **Learning Point:**
    - Observe how the tracker follows the ball's motion.
    - Experiment with parameters like maxCorners (number of features) or winSize (search window size).
- **Discussion:** Can see how parameter tuning affects tracking accuracy and stability.

Mini Project: Motion Tracker (Dense Optical Flow)

- **Workflow:**
    1. Compute dense optical flow using Farneback's algorithm.
    2. Convert flow vectors into magnitude and angle.
    3. Visualize motion using color coding (HSV → BGR).
- **Outcome:** Moving regions are highlighted, giving a clear visualization of motion patterns.
- **Applications:**
    - Detecting unusual movement in surveillance.
    - Analyzing player motion in sports.
    - Visualizing traffic flow in autonomous driving research.

Tips

- Emphasize the **difference between sparse and dense optical flow**: efficiency vs detail.
- Show how **KLT builds on optical flow** by focusing on "good features to track."
- Test with different videos (fast motion, occlusion, multiple objects).
- Connect exercises to **realworld systems**: CCTV tracking, sports replay analysis, driver assistance systems.

Introduction to ML in CV

Why ML in Computer Vision?

- **Definition:** Machine Learning (ML) algorithms learn patterns from data and use them to classify or recognize new inputs.
- **Purpose in CV:** Instead of handcrafted rules, ML models can automatically learn to distinguish visual categories (digits, objects, faces).
- **Applications:**
    - Handwritten digit recognition (MNIST).
    - Object classification in photos.
    - Medical image analysis.

kNearest Neighbors (kNN)

- **Concept:** Classifies a sample based on the majority label of its *k* closest training examples.
- **Steps:**
    1. Compute distance (e.g., Euclidean) between the test sample and all training samples.
    2. Select the *k* nearest neighbors.
    3. Assign the class most common among them.
- **Pros:**
    - Simple, intuitive, no training phase.
    - Works well for small datasets.
- **Cons:**
    - Slow for large datasets (needs to compare with all samples).
    - Sensitive to irrelevant features and choice of distance metric.
- **Exercise Outcome:** Train a kNN classifier on MNIST digits and see classification accuracy.

Support Vector Machines (SVM)

- **Concept:** Finds a hyperplane that best separates classes with maximum margin.
- **Steps:**
    1. Map data into highdimensional space.

2. Find separating hyperplane with largest margin.
3. Classify new samples based on which side of the hyperplane they fall.

- **Pros:**
  - Effective in highdimensional spaces.
  - Robust to overfitting when properly tuned.
- **Cons:**
  - Computationally expensive for large datasets.
  - Requires careful choice of kernel and parameters.
- **Exercise Outcome:** Train an SVM classifier on MNIST digits and compare results with kNN.

Mini Project: Handwritten Digit Recognizer

- **Workflow:**
  1. Train SVM on MNIST dataset.
  2. Write a function to predict the label of a new digit image.
  3. Test with sample digits.
- **Outcome:** Build a simple recognizer that can classify handwritten digits, demonstrating how ML models can be applied to real CV tasks.

Tips

- Connect exercises to **realworld applications**: postal code recognition, bank check processing, OCR systems.

Bag of Visual Words (BoVW)

Why BoVW?

- **Concept:** Inspired by text analysis, where documents are represented as word frequency histograms.
- **In CV:** Local image features (like SIFT or ORB descriptors) are clustered into “visual words.”

- **Purpose:** Converts variablelength feature sets into fixedlength histograms, enabling classification with standard ML algorithms.
- **Applications:** Texture classification, scene recognition, object categorization.

Feature Clustering

- **Step 1:** Extract local features from images (e.g., SIFT descriptors).
- **Step 2:** Use clustering (commonly kmeans) to group descriptors into clusters.
- **Step 3:** Each cluster center represents a **visual word**.
- **Analogy:** Just like grouping synonyms in language, clustering groups similar visual patterns.

Histogram Representation

- **Step 1:** Assign each feature in an image to its nearest cluster (visual word).
- **Step 2:** Count occurrences of each word → build a histogram.
- **Step 3:** Histogram becomes the image's representation.
- **Benefit:** Fixedlength vector regardless of how many features were extracted.
- **Use:** These histograms can be fed into classifiers like SVM or kNN.

Exercise: Build BoVW Model for Textures

- **Goal:** Cluster descriptors into visual words and represent each image as a histogram.
- **Learning Point:** See how raw descriptors are transformed into structured vectors.
- **Discussion:** Changing the number of clusters (k) alters the vocabulary size and affects classification accuracy.

Mini Project: Texture Classifier

- **Workflow:**
    1. Extract features from texture images.
    2. Cluster descriptors into visual words.
    3. Build histograms for each image.
    4. Train an SVM classifier on these histograms.
- **Outcome:** Build a classifier that distinguishes textures based on BoVW representation.

- **Applications:** Material recognition, surface inspection, texturebased retrieval.

Tips

- Emphasize the **analogy with text classification**: words ↔ visual words, document ↔ image, word frequency ↔ histogram.
- Show how BoVW bridges **feature extraction** and **machine learning classification**.
- Experiment with different clustering sizes (k) and classifiers.
- Connect exercises to **realworld tasks**: e.g., recognizing fabrics in fashion, categorizing natural textures in environmental monitoring.

Face Detection

Why Face Detection?

- **Definition:** Face detection is the process of locating human faces in images or video.
- **Purpose:** It's a fundamental step before face recognition, emotion analysis, or biometrics.
- **Applications:**
    - Security and surveillance.
    - Human–computer interaction (e.g., face unlock).
    - Social media filters and AR.

Haar Cascades

- **Concept:** Haar features are simple rectangular patterns (like edge detectors).
- **Training:** Uses **AdaBoost** to select the most important features and combine them into a cascade classifier.
- **Pros:**
    - Fast and lightweight.
    - Works well for frontal faces in controlled environments.
- **Cons:**
    - Sensitive to lighting, pose, and occlusion.
    - Less robust for complex scenes.

- **Exercise Outcome:** See bounding boxes around detected faces in static images or webcam feed.

HOG (Histogram of Oriented Gradients)

- **Concept:** Captures gradient orientation distributions in local regions of the image.
- **Usage:** Classical method for detecting faces and pedestrians.
- **Pros:**
    - Robust to illumination changes.
    - Captures shape and edge information effectively.
- **Cons:**
    - Computationally heavier than Haar cascades.
    - May require more preprocessing.
- **Discussion:** Compare Haar vs HOG in terms of speed and robustness.

Exercise: Detect Faces in Webcam Feed

- **Goal:** Run Haar cascade detection live on webcam input.
- **Learning Point:**
    - Observe how detection works in realtime.
    - Notice limitations (false positives, missed detections).
- **Experiment:** Adjust parameters like scaleFactor and minNeighbors in detectMultiScale to improve accuracy.

Mini Project: Face Detector App

- **Workflow:**
    1. Load Haar cascade.
    2. Detect faces in either static images or webcam feed.
    3. Draw bounding boxes around detected faces.
- **Outcome:** Build a simple app that can detect faces interactively.
- **Applications:** Entrylevel prototype for security systems, attendance monitoring, or AR filters.

Tips

- Emphasize the **difference between Haar (fast, lightweight)** and **HOG (robust, heavier)**.
- Connect exercises to **realworld systems**: CCTV monitoring, smartphone face unlock, social media filters.

PreCNN Module

Programming Basics (Python + NumPy)

- **Concepts to cover:**
  - Variables, loops, functions, classes.
  - Arrays and matrices with NumPy.
  - Plotting with Matplotlib.
- **Analogy:** Think of NumPy arrays as Excel sheets where each cell holds a number, but with superpowers for math.

Mathematics Foundations

- **Linear Algebra:**
  - Vectors and matrices.
  - Dot product → similarity measure.
  - Convolution = sliding dot product.
- **Calculus:**
  - Derivatives → slope of a curve.
  - Gradients → direction of steepest change.
- **Probability & Statistics:**
  - Mean, variance, distributions.
  - Why normalization matters.
- **Analogy:** Gradient = “how steep the hill is.” If you’re climbing, the gradient tells you how hard it gets.

Image Basics

- **Pixels & Channels:**
    - Grayscale = one channel.
    - RGB = three channels (red, green, blue).
- **Image as Matrix:**
    - Each pixel = number(s).
- **Transformations:**
    - Scaling, rotation, cropping.
- **Analogy:** Pixels are Lego blocks; arranging them builds the image.
- **Activity:** Rotate and flip an image in code, then discuss how transformations affect recognition.

Machine Learning Fundamentals

Machine Learning (ML) is a branch of artificial intelligence that enables computers to learn from data and improve their performance without being explicitly programmed. Instead of writing rules manually, we give the computer examples and it figures out the patterns on its own.

Supervised learning is one of the simplest and most common types of machine learning.

What It Is?

- In supervised learning, the computer learns from examples that already have answers (labels).
- You give the algorithm a dataset with inputs (features) and outputs (labels).
- The goal is for the algorithm to learn the relationship between inputs and outputs so it can predict the output for new, unseen inputs.

Example

- Suppose you want to teach a model to recognize fruits.
- Inputs: Features like color, size, shape.
- Labels: “Apple,” “Banana,” “Orange.”

- The model sees many examples and learns patterns.
- Later, when you show it a new fruit, it predicts the label based on what it learned.

Key Points

- Training data: Must include both inputs and outputs.
- Prediction: Model tries to guess the output for new inputs.
- Evaluation: Accuracy is measured by comparing predictions with true labels.
- Examples of supervised learning tasks:
    - Classification (e.g., spam vs not spam emails).
    - Regression (e.g., predicting house prices).

Supervised learning = learning with guidance. The model is trained on labeled data so it can make predictions on new data.

- **Supervised Learning:**
    - Input → label → prediction.
- **Basic Models:**
    - kNN: “majority vote of neighbours.”
    - SVM: “find the best dividing line.”

Unsupervised learning is a type of machine learning where the computer learns patterns from data without any labels or predefined answers.

What It Is?

- In supervised learning, you give the model inputs with labels (e.g., “this is a cat”).
- In unsupervised learning, you only give the model inputs, and it has to discover structure by itself.
- The goal is to find hidden patterns, groupings, or relationships in the data.

Example

Imagine you have a basket of fruits but no labels:

- The algorithm looks at features like color, size, and shape.

- It might group red, round fruits together (apples), and long, yellow ones together (bananas).
- You didn't tell it what an apple or banana is — it figured out clusters on its own.

Key Techniques

- Clustering: Grouping similar data points (e.g., kmeans, hierarchical clustering).
- Dimensionality Reduction: Simplifying data while keeping important patterns (e.g., PCA, tSNE).
- Association: Finding rules or relationships (e.g., "people who buy bread also buy butter").

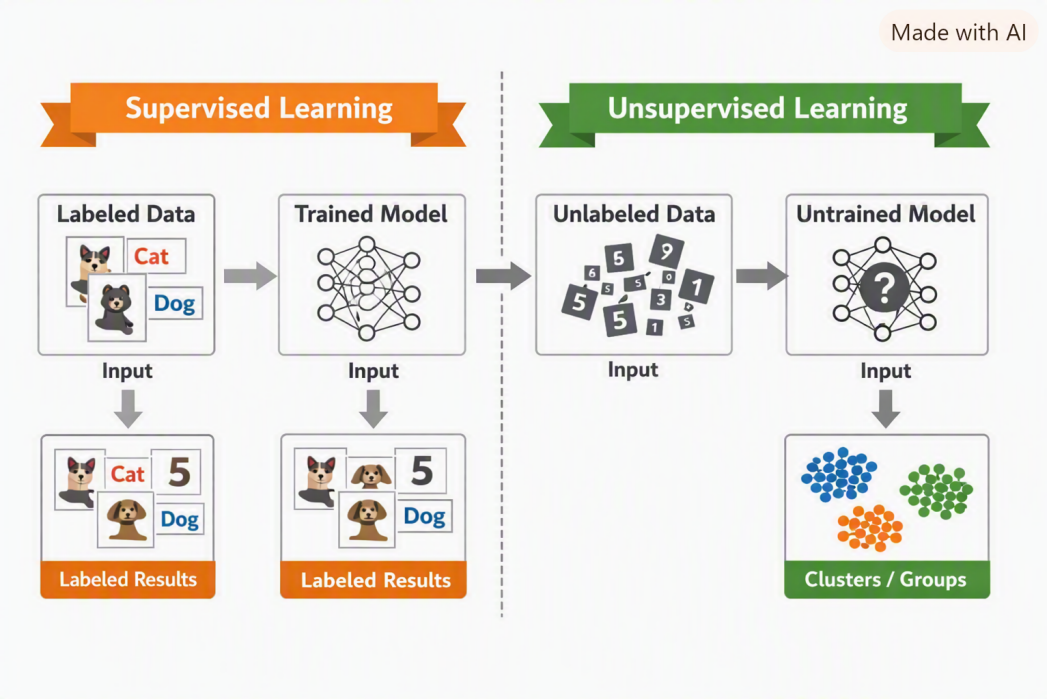
Applications

- Customer segmentation in marketing.
- Organizing large image collections.
- Detecting anomalies (fraud detection, unusual behavior).
- Recommendation systems (grouping similar users or products).

In short: unsupervised learning = learning without guidance. The model finds patterns and groups in unlabeled data.

Aspect	**Supervised Learning**	**Unsupervised Learning**
Definition	Learns from labeled data (inputs + correct outputs).	Learns from unlabelled data (only inputs, no answers).
Goal	Predict outcomes for new data based on training with known labels.	Discover hidden patterns, groupings, or structure in data.
Data Requirement	Requires labeled dataset (e.g., images with categories, emails marked spam/not spam).	Works with raw, unlabeled dataset (e.g., customer purchase records without categories).
Techniques	Classification (spam detection, digit recognition), Regression (predicting house prices).	Clustering (kmeans, hierarchical), Dimensionality Reduction (PCA, tSNE), Association rules.
Output	Specific prediction (class label or numeric value).	Groups, clusters, or reduced representation of data.

Aspect	Supervised Learning	Unsupervised Learning
Analogy	Like a teacher giving students questions with answers to learn from.	Like exploring a new city without a guide, discovering neighbourhoods yourself.
Examples	- Handwritten digit recognition (MNIST). - Predicting medical diagnoses. - Sentiment analysis of reviews.	- Customer segmentation in marketing. - Organizing photo collections. - Detecting anomalies in transactions.



ANN

An artificial neuron is the basic computational unit in an Artificial Neural Network (ANN), inspired by biological neurons but implemented mathematically. It's the building block that allows machine learning models to learn patterns from data.

Structure of an Artificial Neuron

- Inputs $(x_1, x_2, \ldots, x_n)$: Features or signals fed into the neuron.
- Weights $(w_1, w_2, \ldots, w_n)$: Each input has an associated weight that determines its importance.
- Bias (b): A constant added to shift the activation function.
- Summation: The neuron computes a weighted sum:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Activation Function ($f(z)$): Introduces nonlinearity, deciding whether the neuron "fires." Common functions: ReLU, Sigmoid, Tanh.
- Output: The result is passed to the next layer of neurons.

How It Works?

1. Inputs (like pixel values of an image) enter the neuron.
2. Each input is multiplied by its weight.
3. The weighted sum plus bias is calculated.
4. The activation function transforms this sum into an output.
5. The output is sent forward to other neurons.

Analogy

Think of an artificial neuron as a decision unit:

- Inputs = evidence.
- Weights = importance of each piece of evidence.
- Bias = prior tendency.
- Activation = final decision (yes/no, strong/weak).

Example

For digit recognition:

- Inputs = pixel values of a handwritten "5."
- Weights = learned importance of certain strokes.
- Activation = decides whether the image resembles "5."
- Output = probability of being digit "5."

In short: an artificial neuron is a mathematical model that mimics how biological neurons process signals, enabling ANNs to learn and make predictions.

Neural Network Basics (ANN)

- **Artificial Neurons:**
    - Weighted sum + activation function.
- **Layers:**
    - Input → hidden → output.
- **Forward Pass:**
    - Data flows forward, predictions made.
- **Backpropagation:**
    - Errors flow backward, weights updated.
- **Analogy:** Each neuron is like a student giving a small opinion; the network combines them into a final decision.
- **Activity:** Build a small feedforward network to classify digits, visualize accuracy improvement

Bridge to CNNs

- **Problem with Fully Connected Networks:**
    - Too many parameters for large images.
    - Lose spatial structure (pixels treated independently).
- **Motivation for CNNs:**
    - Convolutions preserve spatial locality.
    - Filters detect edges, textures, shapes.
- **Analogy:** CNNs are like detectives using magnifying glasses (filters) to spot clues (features) in different parts of the image.
- **Activity:** Apply a simple convolution filter (edge detector) manually, then show how CNNs automate this process.
- Build **progressive complexity**: start with pixels → features → ML → ANN → CNN.

CNN Basics

Why CNNs?

- **Definition:** Convolutional Neural Networks (CNNs) are specialized neural networks designed for image data.
- **Purpose:** They automatically learn spatial hierarchies of features — from edges and textures in early layers to complex shapes in deeper layers.
- **Applications:** Image classification, object detection, face recognition, medical imaging.

Convolutions

- **Concept:** Convolution applies small filters (kernels) across the image to detect local patterns.
- **What They Capture:**
    - Early layers → edges, corners, textures.
    - Deeper layers → shapes, objects, semantic features.
- **Benefit:** Reduces parameters compared to fully connected layers, making CNNs efficient and scalable.

Pooling

- **Concept:** Pooling down samples feature maps by summarising regions.
- **Types:**
    - **Max Pooling:** Takes the maximum value in each region (captures strongest features).
    - **Average Pooling:** Takes the average (smooths features).
- **Purpose:**
    - Reduces computation.
    - Provides translation invariance (robust to small shifts).
    - Controls overfitting.

Activation Functions

- **Concept:** Introduce nonlinearity so the network can learn complex patterns.
- **Common Functions:**
    - **ReLU (Rectified Linear Unit):** Fast, effective, widely used.
    - **Sigmoid:** Maps values between 0 and 1, useful for probabilities.
    - **Tanh:** Maps values between -1 and 1, but less common today.
- **Role:** Without activation functions, CNNs would behave like linear models and fail to capture complex relationships.

Exercise: Implement CNN in PyTorch for MNIST

- **Goal:** Build a CNN that classifies handwritten digits.
- **Architecture:**
    - Two convolutional layers (Conv2d).
    - Pooling and dropout for regularization.
    - Fully connected layers for classification.
- **Learning Point:** See how CNN layers transform raw pixel data into meaningful features.

Training & Testing Loop

- **Training:**
    - Forward pass → compute predictions.
    - Loss function (negative log likelihood).
    - Backpropagation → update weights using optimizer (Adam).
- **Testing:**
    - Evaluate on unseen data.
    - Report accuracy and loss.
- **Outcome:** Observe how accuracy improves with training epochs.

Mini Project: Digit Classifier

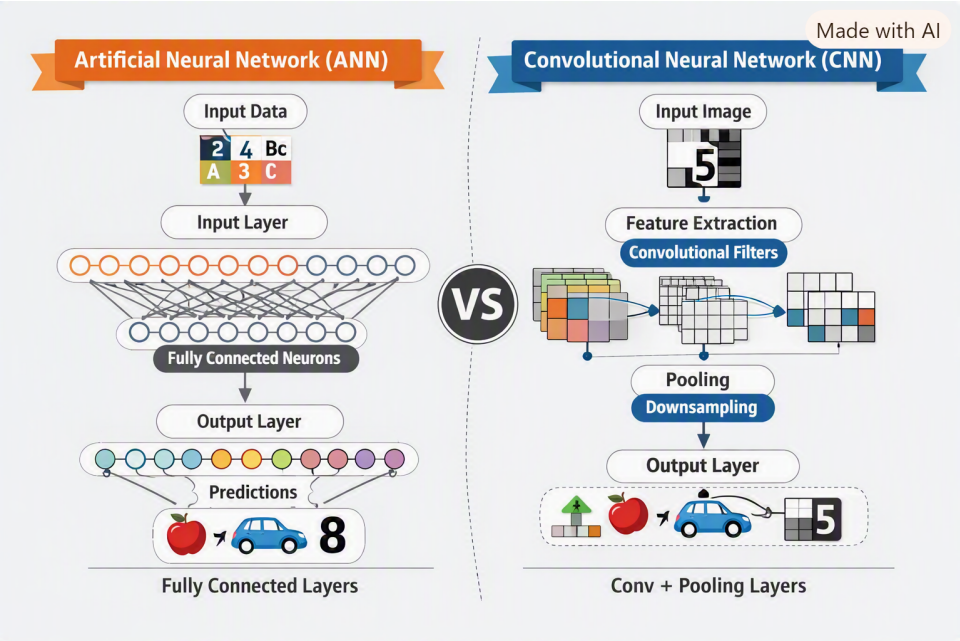
- **Workflow:**

1. Train CNN on MNIST dataset.
2. Test with custom digit images or drawings.
3. Evaluate predictions.

- **Outcome:** Build a practical digit recognizer, connecting theory to application.
- **Applications:** OCR (Optical Character Recognition), postal code recognition, bank check processing.

Tips

- Emphasize the **flow of data**: Input → Convolution → Pooling → Flatten → Fully Connected → Output.
- Connect exercises to **realworld systems**: digit recognition apps, face unlock, medical image classifiers.



What are Keypoints?

In computer vision, **keypoints** are the specific locations in an image that an algorithm identifies as being *distinctive and informative*. They are essentially the "coordinates" of features.

- **Definition:** A keypoint is a point in the image (x, y coordinate) that marks a feature such as a corner, blob, or edge.
- **Role:** They serve as anchors for describing and matching images.

- **Example:** In SIFT or ORB, each detected feature is represented by a keypoint object that stores:
    - Position (x, y)
    - Scale (size of the feature)
    - Orientation (direction of the gradient)
    - Response (strength of detection)

Why Keypoints Matter?

- **Repeatability:** The same keypoint should be detected even if the image is rotated, scaled, or slightly changed.
- **Matching:** Keypoints allow us to compare two images by finding correspondences between their distinctive points.
- **Efficiency:** Instead of comparing every pixel, we only compare descriptors at keypoints.

Examples

- **Harris Corners:** Keypoints are corners where intensity changes in two directions.
- **SIFT:** Keypoints are scale and rotationinvariant points, each with a descriptor vector.

ORB: Keypoints are FAST corners with orientation and BRIEF descriptors.

What are Feature Maps?

- **Definition:** A feature map is a 2D grid (matrix) of values that represents how strongly a particular filter responds to different regions of the input.
- **Process:**
    1. A kernel (small filter, e.g., 3×3) slides over the image.
    2. At each position, it computes a weighted sum of pixel values.
    3. The result is stored in the feature map.

What Pooling Does?

- **Definition:** Pooling summarizes regions of a feature map into a single value.
- **Purpose:**
    - Reduces dimensionality (smaller feature maps).

    - Makes the network less sensitive to small translations or distortions.
    - Speeds up computation and reduces overfitting.

What Stride Means?

- **Definition:** The step size of the kernel when sliding over the image.
- **Effect:** Controls how much the receptive fields overlap and determines the output size of the feature map.

What Padding Does?

- **Definition:** Padding is the process of surrounding the input image with additional pixels (usually zeros).
- **Purpose:**
    - Prevents the image from shrinking after each convolution.
    - Ensures that filters can cover edge pixels.
    - Helps control the output size of feature maps.

What Flatten Does?

- **Definition:** Flatten reshapes the output of convolutional/pooling layers into a single long vector.
- **Purpose:**
    - Fully connected layers expect a 1D input.
    - Flatten bridges the transition from spatial feature maps to classification layers.